INFORME DE LA GUÍA DE APRENDIZAJE N° 6

Fundamentos de SQL y MySQL — Proyecto GameStore Escolar

Unidad didáctica: Taller de Lenguajes de Programación

Estudiante: Marcos Riquelme Huahuiza Sarayasi
Institución: Instituto Superior Tecnológico de La Joya

1. Objetivo

Diseñar, crear y gestionar una base de datos relacional en MySQL mediante comandos SQL, para estructurar la información del sistema GameStore Escolar y garantizar la integridad referencial entre sus tablas.

2. Herramientas utilizadas

- XAMPP, que incluye el servidor MySQL/MariaDB y Apache.
- phpMyAdmin, como cliente para administrar la base de datos y ejecutar las consultas.
- Base de datos: gamestore_db.

3. Trabajo 1: Diseño del esquema relacional (DDL)

Se creó la base de datos gamestore_db con tres tablas relacionadas: estudiantes, recompensas e historial_canjes. Esta última tiene dos llaves foráneas que apuntan a las otras dos tablas.

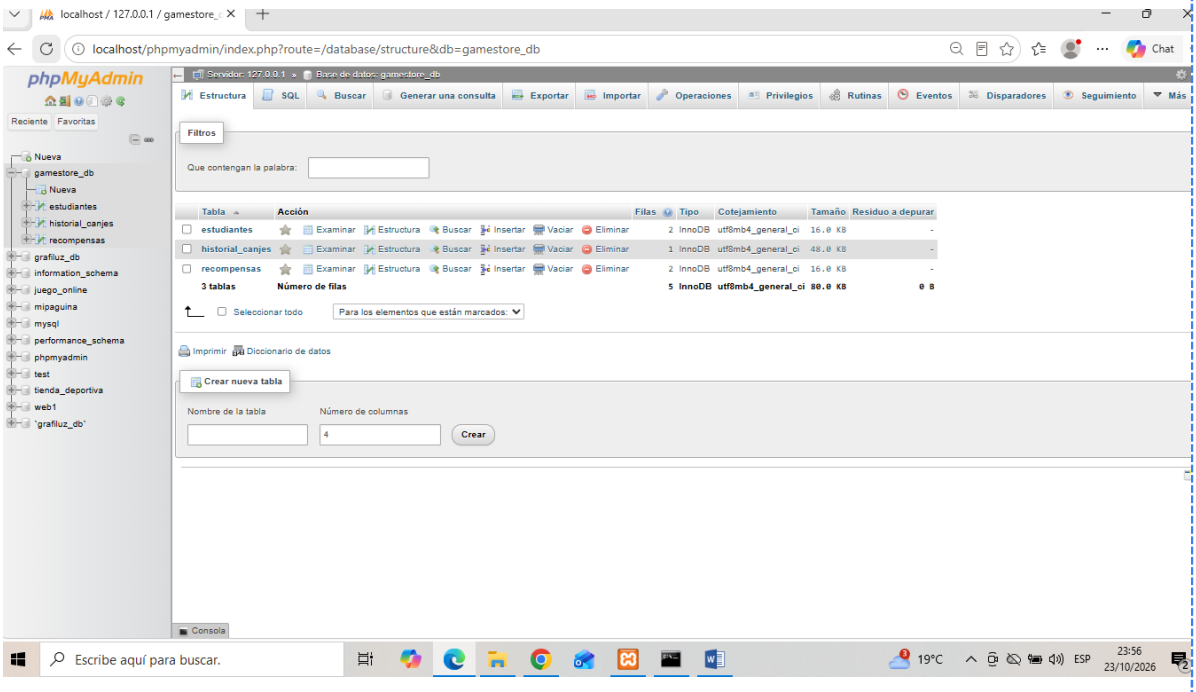
```
CREATE DATABASE IF NOT EXISTS gamestore_db;
USE gamestore_db;

CREATE TABLE estudiantes (
    id_estudiante INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    puntos INT DEFAULT 0
);

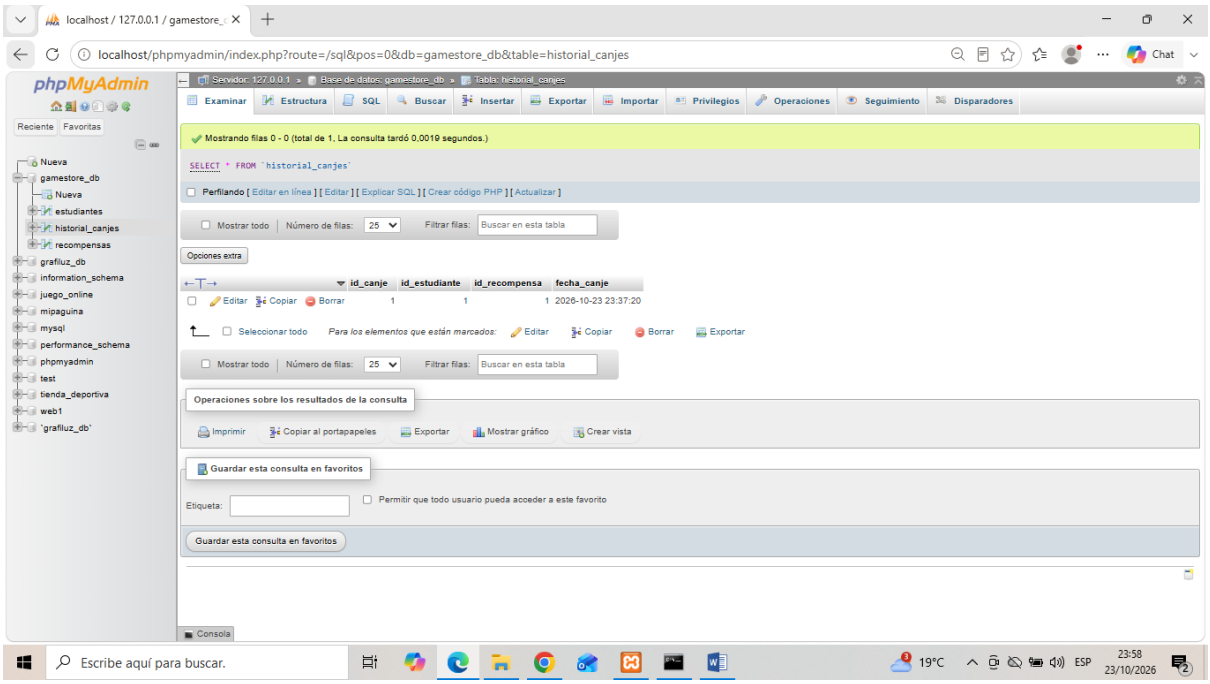
CREATE TABLE recompensas (
    id_recompensa INT AUTO_INCREMENT PRIMARY KEY,
    nombre_item VARCHAR(100) NOT NULL,
    costo_puntos INT NOT NULL
);

CREATE TABLE historial_canjes (
    id_canje INT AUTO_INCREMENT PRIMARY KEY,
    id_estudiante INT,
    id_recompensa INT,
    fecha_canje TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_estudiante) REFERENCES estudiantes(id_estudiante),
    FOREIGN KEY (id_recompensa) REFERENCES recompensas(id_recompensa)
);
```

Captura 1: Tablas creadas en gamestore_db (lista de las 3 tablas en phpMyAdmin)



Captura 2: Vista de relaciones de historial_canjes (Estructura → Vista de relaciones, con las dos llaves foráneas)



4. Trabajo 2: Ejecución de consultas SQL (DML)

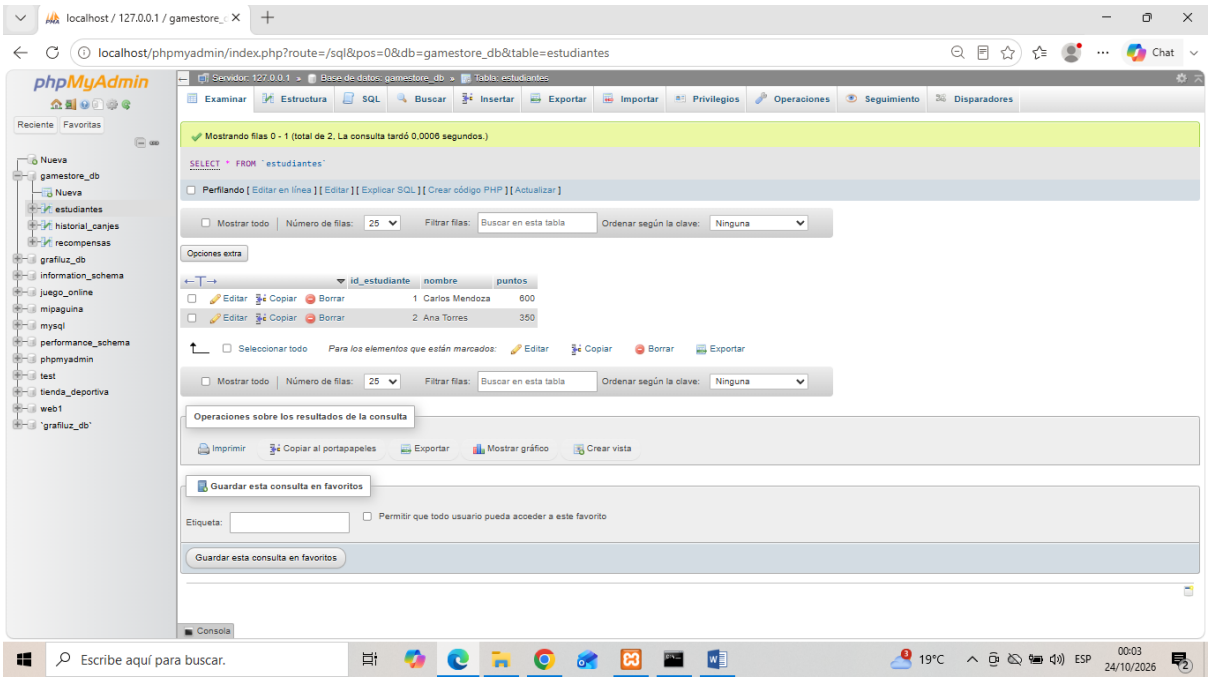
Se insertaron datos de prueba, se consultaron, se actualizó un puntaje y se registró un canje relacionando las tablas.

```
INSERT INTO estudiantes (nombre, puntos)
VALUES ('Carlos Mendoza', 500), ('Ana Torres', 350);

INSERT INTO recompensas (nombre_item, costo_puntos)
VALUES ('Skin Guerrero Cyber', 150), ('Pase de Prórroga', 300);

SELECT * FROM estudiantes;
```

Captura 3: SELECT * FROM estudiantes; inicial (Carlos con 500 puntos y Ana con 350)

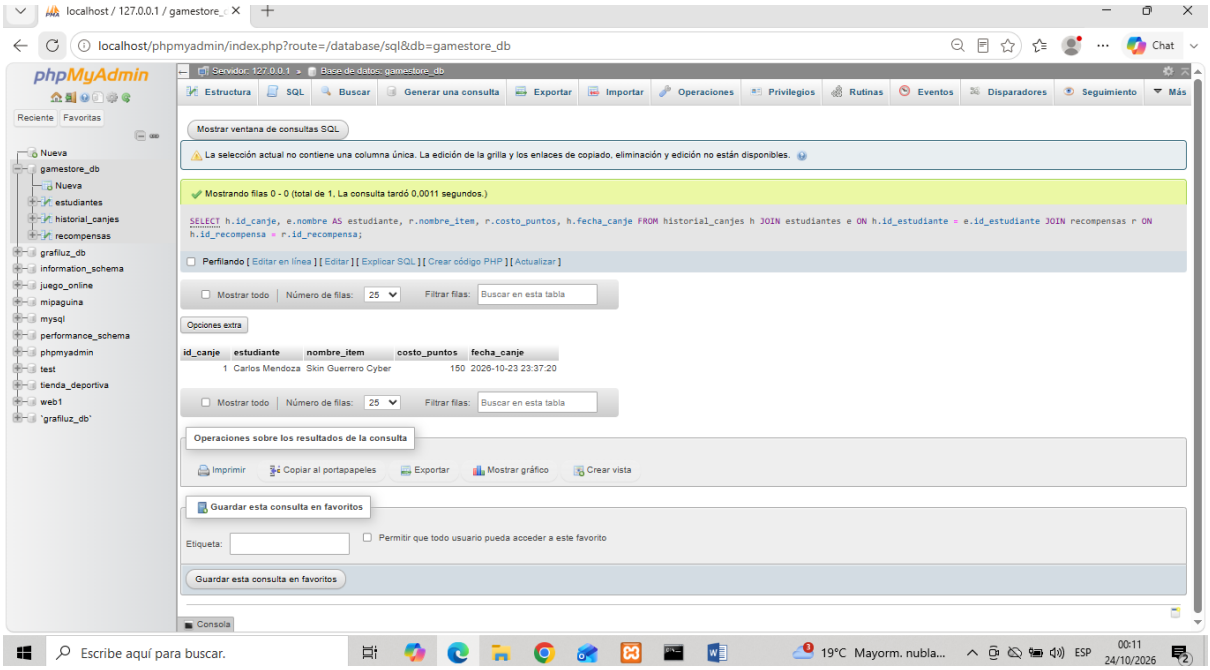



```
UPDATE estudiantes SET puntos = 600 WHERE id_estudiante = 1;

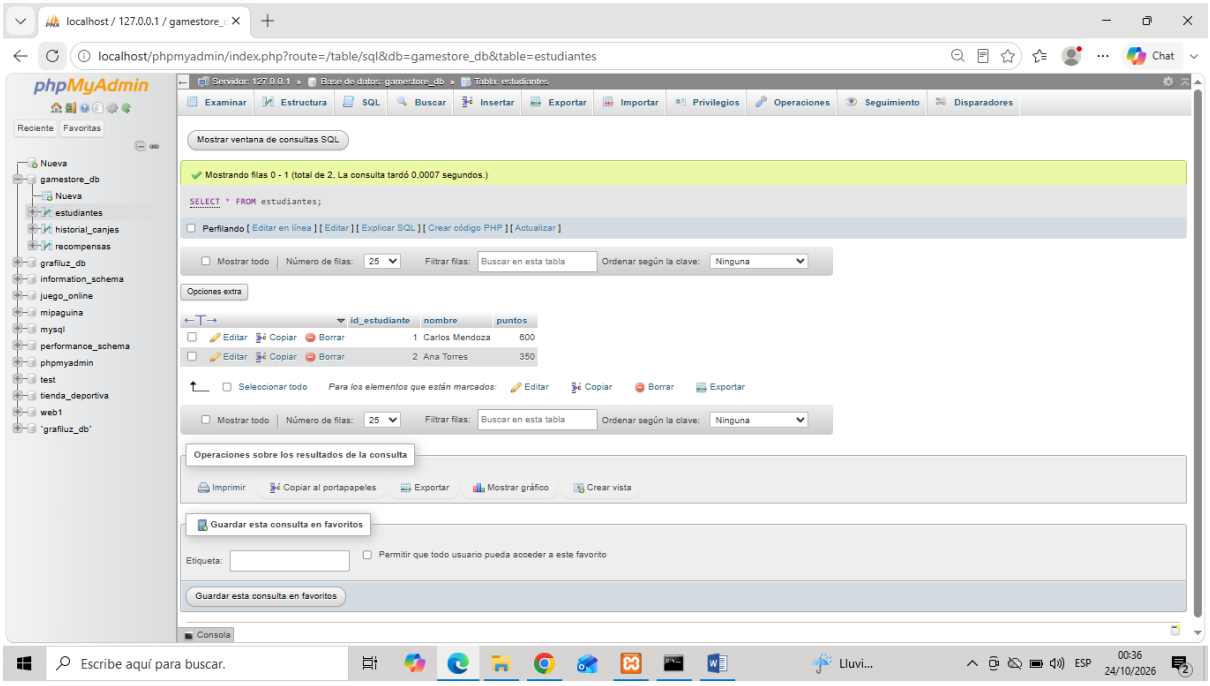
INSERT INTO historial_canjes (id_estudiante, id_recompensa) VALUES (1, 1);

SELECT h.id_canje,
       e.nombre AS estudiante,
       r.nombre_item,
       r.costo_puntos,
       h.fecha_canje
FROM historial_canjes h
JOIN estudiantes e ON h.id_estudiante = e.id_estudiante
JOIN recompensas r ON h.id_recompensa = r.id_recompensa;
```

Captura 4: Resultado del JOIN (historial detallado de canjes)



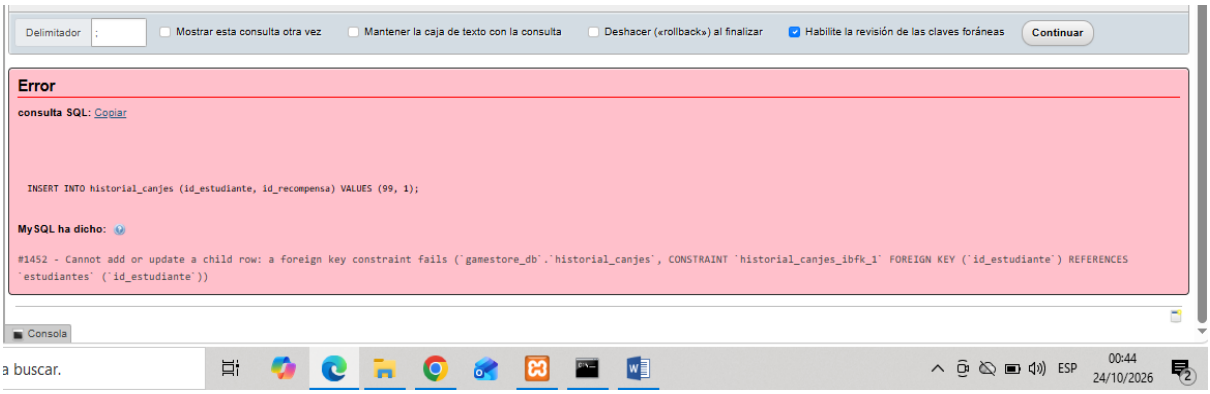
Captura 5: SELECT * FROM estudiantes; final (Carlos ya con 600 puntos tras el UPDATE)



5. Sustentación conceptual: función de la llave foránea

Una llave foránea (FOREIGN KEY) es una columna que apunta a la llave primaria de otra tabla. Su función es garantizar la integridad referencial: MySQL no permite registrar un valor que no exista en la tabla relacionada (por ejemplo, un canje de un estudiante o de una recompensa inexistentes) ni borrar un registro que otro está usando. De esta forma se evitan los datos huérfanos y se conectan las tablas sin duplicar información.

Captura 6: (Opcional) Error #1452 al insertar un estudiante inexistente: INSERT INTO historial_canjes (id_estudiante, id_recompensa) VALUES (99, 1);



6. Conclusiones

- Se creó correctamente la base de datos gamestore_db con sus tres tablas, sus llaves primarias y sus llaves foráneas.
- Los comandos INSERT, SELECT y UPDATE permitieron registrar, consultar y modificar la información del sistema.
- El JOIN permitió unir las tres tablas y mostrar el historial de canjes con los nombres en lugar de los identificadores.
- Las llaves foráneas protegen la integridad de los datos al impedir registros que apunten a filas inexistentes.

7. Evidencia adjunta

- Script SQL completo: sesion6_gamestore.sql